

Claude CLI PPT Automation

Day7 전체 파일 샘플 모음

강의 스크립트 없이 파일별 샘플 코드만 수록

파일 목록

01. CLAUDE.md
02. README.md
03. .gitignore
04. .claude/settings.json
05. .claude/settings.local.json
06. .claude/rules/project-architecture.md
07. .claude/rules/presentation-schema.md
08. .claude/rules/naming-convention.md
09. .claude/rules/design-system.md
10. .claude/rules/output-policy.md
11. .claude/agents/slide-planner.md
12. .claude/agents/slide-designer.md
13. .claude/agents/slide-builder.md
14. .claude/agents/slide-reviewer.md
15. .claude/skills/presentation-schema/SKILL.md
16. .claude/skills/presentation-schema/reference.md
17. .claude/skills/presentation-schema/examples/example-page.md
18. .claude/skills/design-tokens/SKILL.md
19. .claude/skills/design-tokens/reference.md
20. .claude/skills/create-slide/SKILL.md
21. .claude/skills/create-slide/templates/page-template.md
22. .claude/skills/build-presentation/SKILL.md
23. .claude/skills/validate-presentation/SKILL.md
24. .claude/skills/validate-presentation/checklist.md
25. .claude/hooks/validate-page-id.ps1
26. .claude/hooks/validate-page-schema.ps1
27. .claude/hooks/validate-layout.ps1
28. .claude/hooks/validate-build.ps1
29. presentation/config/document.md
30. presentation/config/page-types.md
31. presentation/config/manifest.md
32. presentation/design-system/typography.md
33. presentation/design-system/colors.md
34. presentation/design-system/spacing.md
35. presentation/design-system/image-boxes.md
36. presentation/design-system/positions.md
37. presentation/design-system/layouts.md
38. presentation/pages/p1.md
39. presentation/pages/ps1.md
40. presentation/pages/ps1_1.md
41. presentation/pages/ps1_2.md
42. presentation/pages/ps1_3.md
43. presentation/pages/pb1.md
44. presentation/pages/pb1_1.md
45. presentation/pages/pb1_2.md
46. presentation/pages/pb1_3.md

- 47. renderer/templates/cover.html
- 48. renderer/templates/section.html
- 49. renderer/templates/content.html
- 50. renderer/css/tokens.css
- 51. renderer/css/components.css
- 52. renderer/css/slides.css
- 53. renderer/js/presentation.js
- 54. renderer/index.html
- 55. scripts/build.ps1
- 56. scripts/export-pdf.ps1
- 57. scripts/validate.ps1

01. CLAUDE.md

```
# PPT Automation Project

## Purpose
구조화된 Markdown 페이지 데이터를 기반으로 1920×1080 프레젠테이션을 생성한다.

## Source of Truth
- `presentation/pages`: 페이지 원본
- `presentation/design-system`: 디자인 토큰
- `presentation/config/manifest.md`: 페이지 등록 순서와 기본 속성
- `output`: 결과물, 직접 수정 금지

## Core Workflow
사용자 요청 → 페이지/요소 식별 → 필요한 Agent/Skill 사용 → 원본 수정 → 검증 → 빌드 → 출력

## Core Rules
- 페이지 ID를 임의 변경하지 않는다.
- 기존 디자인 토큰을 우선 사용한다.
- 특정 페이지 요청 시 다른 페이지를 수정하지 않는다.
- 특정 요소 요청 시 해당 요소만 수정한다.
- `output` 파일을 직접 수정하지 않는다.
- 렌더링 결과는 원본 MD를 수정하여 변경한다.
- 위치 속성이 충돌하면 X/Y 절대 좌표를 최우선으로 적용한다.

## Page IDs
- `p1`: 표지
- `ps1`: 서론 대표 페이지
- `ps1_1` ~ `ps1_3`: 서론 상세 페이지
- `pb1`: 본론 대표 페이지
- `pb1_1` ~ `pb1_3`: 본론 상세 페이지

## Element IDs
페이지 내부 요소는 `{pageId}.{elementId}` 형식을 사용한다.

예:
- `ps1_2.title`
- `ps1_2.body01`
- `ps1_2.image01`
- `pb1_3.box01`
```

02. README.md

```
# Day7 - Claude CLI PPT Automation
```

Claude Code 의 Rules, Subagents, Skills, Hooks 를 사용해 구조화된 PPT 제작 파이프라인을 실습하는 프로젝트다.

```
## 시작
```

```
```powershell
cd C:\Agent\Day7
claude
```
```

```
## 주요 디렉터리
```

```
- `.claude/rules` : 프로젝트 상시 규칙
- `.claude/agents` : 역할별 Subagent
- `.claude/skills` : 반복 작업 절차
- `.claude/hooks` : 자동 검증 PowerShell
- `presentation` : 프레젠테이션 원본 데이터
- `renderer` : HTML 렌더러
- `scripts` : 빌드/검증/내보내기 스크립트
- `output` : 생성 결과
```

```
## 기본 작업 순서
```

1. `presentation/config/manifest.md` 확인
2. `presentation/pages/\*.md` 생성 또는 수정
3. `scripts/validate.ps1` 실행
4. `scripts/build.ps1` 실행
5. 필요 시 `scripts/export-pdf.ps1` 실행

```
## 확인 명령
```

```
```powershell
tree /F
powershell -ExecutionPolicy Bypass -File .\scripts\validate.ps1
```
```

03. .gitignore

```
# Local Claude settings
.claude/settings.local.json

# Generated output
output/html/*
output/pdf/*
output/pptx/*

# Logs
logs/*

# Temporary files
*.tmp
*.bak
~$*

# OS / editor
.DS_Store
Thumbs.db
.vscode/
```

04. .claude/settings.json

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "powershell -NoProfile -ExecutionPolicy Bypass -File \"${CLAUDE_PROJECT_DIR}\\\\.claude\\hooks\\validate-page-schema.ps1\""
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "powershell -NoProfile -ExecutionPolicy Bypass -File \"${CLAUDE_PROJECT_DIR}\\scripts\\validate.ps1\""
          }
        ]
      }
    ]
  }
}
```

05. .claude/settings.local.json

```
{
  "permissions": {
    "allow": [
      "Bash(powershell -NoProfile -ExecutionPolicy Bypass -File .\\scripts\\validate.ps1)",
      "Bash(powershell -NoProfile -ExecutionPolicy Bypass -File .\\scripts\\build.ps1)"
    ]
  }
}
```


06. .claude/rules/project-architecture.md

Project Architecture Rules

기본 구조

- Claude 설정은 `.claude/`에서 관리한다.
- 프레젠테이션 원본은 `presentation/`에서 관리한다.
- 렌더링 관련 코드는 `renderer/`에서 관리한다.
- 자동화 스크립트는 `scripts/`에서 관리한다.
- 생성 결과는 `output/`에만 저장한다.

Source of Truth

`presentation/pages`와 `presentation/design-system`을 원본으로 간주한다.

수정 원칙

- 결과물에서 직접 역수정하지 않는다.
- 공통 값 변경은 디자인 시스템에서 수정한다.
- 페이지 고유 값만 해당 페이지 파일에서 수정한다.
- 사용자 요청 범위를 넘어 구조를 임의 변경하지 않는다.

07. .claude/rules/presentation-schema.md

Presentation Schema Rules

Page ID

허용 예:

- `p1`
- `ps1`
- `ps1\_1`
- `pb1`
- `pb1\_3`

금지 예:

- `ps1\_2\_gray`
- `pb1\_img\_right`
- `page-final-new`

Element ID

페이지 내부 요소는 `{pageId}.{elementId}` 형식을 사용한다.

예:

- `ps1\_2.title`
- `ps1\_2.body01`
- `ps1\_2.image01`

Page File Required Fields

모든 페이지 파일은 front matter 에 아래 필드를 가진다.

```
```yaml
```

```
id: ps1_2
```

```
type: content
```

```
background: surface-soft
```

```
layout: image-right
```

```
```
```

원칙

ID 는 위치/역할 식별에 사용하고 색상, 정렬, 크기 같은 스타일 속성을 ID 에 포함하지 않는다.

08. .claude/rules/naming-convention.md

```
# Naming Convention

## 파일명
- Markdown: kebab-case 또는 정의된 Page ID 사용
- PowerShell: kebab-case.ps1
- CSS/JS: kebab-case 또는 기능명 기준

## Page
- 표지: `p{n}`
- 서론: `ps{n}`, `ps{n}_{sub}`
- 본론: `pb{n}`, `pb{n}_{sub}`

## Element
- 제목: `title`
- 부제목: `subtitle`
- 본문: `body01`, `body02`
- 이미지: `image01`, `image02`
- 박스: `box01`, `box02`

## Token
- 색상: `surface-page`, `text-primary`
- 글자: `title`, `h1`, `h2`, `body1`, `body3`
- 이미지: `img-xl`, `img-lg`, `img-md`
- 레이아웃: `cover`, `section`, `image-right`
```

09. .claude/rules/design-system.md

Design System Rules

기본 원칙

- 페이지마다 임의의 값을 반복 작성하지 않는다.
- 먼저 `presentation/design-system/`의 기존 토큰을 검색한다.
- 기존 토큰으로 해결되지 않는 경우에만 새 토큰 추가를 제안한다.

Typography

`presentation/design-system/typography.md` 참조

Colors

`presentation/design-system/colors.md` 참조

Spacing

`presentation/design-system/spacing.md` 참조

Images

`presentation/design-system/image-boxes.md` 참조

Position Priority

1. X/Y 절대 좌표
2. 복합 정렬 (`top-right` 등)
3. 단일 정렬 (`left`, `center`, `right`)
4. 레이아웃 기본값

10. .claude/rules/output-policy.md

Output Policy

생성 위치

- HTML: `output/html/`
- PDF: `output/pdf/`
- PPTX: `output/pptx/`
- 로그: `logs/`

금지

- `output/` 내부 파일 직접 편집 금지
- 생성 결과를 Source of Truth로 사용 금지
- 원본과 출력물을 같은 경로에 저장 금지

수정 흐름

잘못된 결과 발견 → 원본 MD 또는 디자인 토큰 수정 → 검증 → 다시 빌드

완료 기준

- 모든 페이지 ID가 유일하다.
- 필수 front matter가 존재한다.
- 1920×1080 범위를 벗어난 요소가 없다.
- 빌드 스크립트가 오류 없이 종료된다.

11. .claude/agents/slide-planner.md

```
----
name: slide-planner
description: PPT 콘텐츠를 분석하고 페이지 구조와 페이지 ID를 계획할 때 사용한다.
tools: Read, Grep, Glob
model: sonnet
skills:
  - presentation-schema
----

# Role
프레젠테이션 정보구조 설계자다.

# Responsibilities
1. 사용자 원고를 페이지 단위로 분할한다.
2. 기존 `manifest.md`와 Page ID를 확인한다.
3. 새로운 페이지가 필요한지 판단한다.
4. 각 페이지의 목적, 유형, 핵심 콘텐츠를 정리한다.

# Restrictions
- 페이지 파일을 직접 수정하지 않는다.
- 기존 Page ID를 임의 변경하지 않는다.
- 디자인 값은 확정하지 않는다.

# Output
페이지 계획을 `ID / Type / Purpose / Required Elements` 형식으로 전달한다.
```

12. .claude/agents/slide-designer.md

```
---
name: slide-designer
description: 페이지 레이아웃, 타이포그래피, 이미지 박스, 정렬과 좌표를 결정할 때 사용한다.
tools: Read, Grep, Glob
model: sonnet
skills:
  - design-tokens
  - presentation-schema
---
```

Role
프레젠테이션 디자인 시스템 설계자다.

Responsibilities

1. 기존 디자인 토큰을 우선 검색한다.
2. 페이지 유형에 적합한 레이아웃을 선택한다.
3. 제목/본문/이미지 토큰을 선택한다.
4. 필요한 경우 좌표와 겹침 규칙을 제안한다.

Restrictions

- 기존 토큰이 있는데 새 값을 임의 생성하지 않는다.
- 콘텐츠의 의미를 변경하지 않는다.
- 실제 페이지 파일 수정은 slide-builder에게 넘긴다.

13. .claude/agents/slide-builder.md

```
---
name: slide-builder
description: 확정된 설계에 따라 페이지 원본과 렌더러를 생성 또는 수정할 때 사용한다.
tools: Read, Write, Edit, Grep, Glob, Bash
model: sonnet
skills:
  - create-slide
  - build-presentation
  - presentation-schema
---
```

Role
프레젠테이션 구현 담당자다.

Work Order

1. 대상 Page ID 확인
2. manifest 확인
3. 관련 디자인 토큰 확인
4. 대상 파일만 수정
5. validation 실행
6. build 실행

Restrictions

- `output/` 파일 직접 수정 금지
- 확정된 레이아웃을 임의 재설계하지 않는다.
- 요청하지 않은 페이지를 수정하지 않는다.

14. .claude/agents/slide-reviewer.md

```
----
name: slide-reviewer
description: 페이지 ID, 디자인 토큰, 좌표, overflow와 변경 범위를 검수할 때 사용한다.
tools: Read, Grep, Glob, Bash
model: sonnet
skills:
  - validate-presentation
----
```

Role
프레젠테이션 QA 담당자다.

Review Items

- Page ID / Element ID
- manifest 등록 여부
- 디자인 토큰 사용 여부
- 배경색 예외 적용 여부
- 1920×1080 영역 초과 여부
- 이미지 비율과 크기
- 수정 대상 이외 파일 변경 여부
- build 결과 존재 여부

Restriction
검수 중 파일을 직접 수정하지 않는다.
문제가 있으면 파일 경로, 요소 ID, 원인을 보고한다.

15. .claude/skills/presentation-schema/SKILL.md

```
---
name: presentation-schema
description: PPT 페이지 ID와 요소 ID, 페이지 front matter 구조를 해석하거나 생성할 때 사용한다.
---
```

Presentation Schema Skill

Before Work

1. `presentation/config/manifest.md`를 읽는다.
2. `reference.md`의 ID 규칙을 확인한다.
3. 기존 페이지 파일에서 동일 ID가 있는지 검색한다.

Page Creation Order

1. Page ID 결정
2. Page Type 결정
3. Background Token 결정
4. Layout Token 결정
5. Element ID 생성
6. 페이지 파일 작성
7. Schema Validation 실행

Important

스타일 속성을 Page ID에 포함하지 않는다.

16. .claude/skills/presentation-schema/reference.md

```
# Presentation Schema Reference

## Page Front Matter

```yaml

id: ps1_2
type: content
background: surface-soft
layout: image-right

```

## Page Types
- `cover`
- `section`
- `content`

## Element Schema

```yaml
id: ps1_2.title
component: h1
text: 프롬프트 엔지니어링
position:
 x: 173
 y: 151
```

## Image Schema

```yaml
id: ps1_2.image01
component: img-xl
source: ../assets/images/example.jpg
align: right
```

## Position Priority
`position.x/y` > compound align > simple align > layout default
```

17. .claude/skills/presentation-schema/examples/example-page.md

```
----
id: ps1_2
type: content
background: surface-soft
layout: image-right
----

# ps1_2

## title
  \yaml
id: ps1_2.title
component: h1
text: 프롬프트 엔지니어링
position:
  x: 173
  y: 151
  \

## body01
  \yaml
id: ps1_2.body01
component: body3
text: AI가 수행해야 할 작업의 조건을 구조적으로 정의합니다.
align: left
  \

## image01
  \yaml
id: ps1_2.image01
component: img-xl
source: ../assets/images/prompt-example.jpg
align: right
  \
```

18. .claude/skills/design-tokens/SKILL.md

```
---
name: design-tokens
description: 프레젠테이션의 타이포그래피, 색상, 간격, 이미지 크기, 위치와 레이아웃 토큰을 선택할 때 사용한다.
---
```

Design Tokens Skill

Workflow

1. `presentation/design-system/` 파일을 검색한다.
2. 요구사항과 가장 가까운 기존 토큰을 선택한다.
3. 토큰 이름과 실제 값을 함께 확인한다.
4. 기존 토큰으로 해결되지 않는 경우에만 새 토큰을 제안한다.

Do Not

- 페이지 안에 임의 HEX 값을 반복 작성하지 않는다.
- 동일 의미의 토큰을 중복 생성하지 않는다.
- 좌표가 있는 요소를 정렬 속성만으로 덮어쓰지 않는다.

세부 목록은 `reference.md`를 확인한다.

19. .claude/skills/design-tokens/reference.md

Design Token Quick Reference

Typography

- `title`: 96px
- `h1`: 60px / 120%
- `h2`: 48px / 120%
- `h3`: 36px / 132%
- `body1`: 36px / 140%
- `body3`: 24px / 134%
- `t1`: 32px / #000000
- `t2`: 24px / #565656

Image Width

- `img-xl`: 640px
- `img-lg`: 420px
- `img-md`: 300px
- `img-sm`: 220px
- `img-xs`: 180px
- `img-2xs`: 150px

Image Boxes

- `ImgBox1`: 952 × 1080
- `ImgBox2`: 824 × 824
- `ImgBox3`: 512 × 678

Surface

- `surface-page`: #FFFFFF
- `surface-soft`: #F1F1F1

20. .claude/skills/create-slide/SKILL.md

```
----
name: create-slide
description: 신규 슬라이드 Markdown 원본을 생성하거나 기존 슬라이드 구조를 확장할 때 사용한다.
----

# Create Slide

## Required Inputs
- Page ID
- Page Type
- Background Token
- Layout Token
- Content

## Procedure
1. manifest 에서 Page ID 중복 확인
2. `templates/page-template.md` 확인
3. Page Type 선택
4. 디자인 토큰 연결
5. 요소 ID 생성
6. `presentation/pages/{pageId}.md` 작성
7. manifest 에 등록
8. validate 실행

## Completion
페이지 파일과 manifest 가 서로 일치해야 한다.
```

21. .claude/skills/create-slide/templates/page-template.md

```
---
id: PAGE_ID
type: content
background: surface-page
layout: default
---

# PAGE_ID

## title
```yaml
id: PAGE_ID.title
component: h1
text: 제목을 입력합니다.
position:
 x: 173
 y: 151
```

## body01
```yaml
id: PAGE_ID.body01
component: body3
text: 본문을 입력합니다.
align: left
```

## image01
```yaml
id: PAGE_ID.image01
component: img-lg
source: ../assets/images/example.jpg
align: right
```
```


22. .claude/skills/build-presentation/SKILL.md

```
----
name: build-presentation
description: presentation 원본을 검증한 뒤 HTML 출력물로 빌드할 때 사용한다.
----

# Build Presentation

## Procedure
1. `scripts/validate.ps1` 실행
2. 오류가 있으면 빌드 중단
3. `scripts/build.ps1` 실행
4. `output/html` 생성 여부 확인
5. 필요한 경우 `scripts/export-pdf.ps1` 실행
6. `validate-build.ps1`로 결과 검증

## Rules
- build 전에 반드시 validate 한다.
- output 을 직접 수정하지 않는다.
- 빌드 결과가 이상하면 원본 또는 renderer 를 수정한 뒤 다시 빌드한다.

## Scripts Directory
복잡한 빌드 작업이 추가되면 이 Skill 의 `scripts/`에 보조 스크립트를 분리한다.
```

23. .claude/skills/validate-presentation/SKILL.md

```
---
name: validate-presentation
description: 프레젠테이션의 페이지 ID, schema, layout, build 결과를 검수할 때 사용한다.
---
```

```
# Validate Presentation
```

```
## Automated Validation
```

```
``powershell
```

```
powershell -ExecutionPolicy Bypass -File .\scripts\validate.ps1
```

```
``
```

```
## Review Order
```

```
1. Page ID
```

```
2. Required Schema
```

```
3. Design Token
```

```
4. Layout Bounds
```

```
5. Manifest Order
```

```
6. Build Output
```

```
7. Requested Change Scope
```

```
## Reporting
```

```
문제 발견 시 다음 형식으로 보고한다.
```

```
- File:
```

```
- Page ID:
```

```
- Element ID:
```

```
- Problem:
```

```
- Recommended Fix:
```

```
세부 항목은 `checklist.md`를 사용한다.
```

24. .claude/skills/validate-presentation/checklist.md

```
# Presentation Validation Checklist

## Schema
- [ ] 모든 Page ID 가 유일하다.
- [ ] 파일명과 front matter 의 ID 가 일치한다.
- [ ] 모든 페이지에 type 이 존재한다.
- [ ] background token 이 정의되어 있다.
- [ ] layout token 이 정의되어 있다.

## Design
- [ ] 기존 typography token 을 사용했다.
- [ ] 임의 색상값이 반복되지 않는다.
- [ ] 이미지 크기가 정의된 token 을 사용한다.
- [ ] 절대 좌표가 문서 범위를 벗어나지 않는다.

## Change Scope
- [ ] 요청한 페이지 외 파일이 변경되지 않았다.
- [ ] output 을 직접 수정하지 않았다.

## Build
- [ ] `output/html/index.html`이 존재한다.
- [ ] build 과정에서 오류가 없다.
```

25. .claude/hooks/validate-page-id.ps1

```
$ErrorActionPreference = "Stop"
$root = Join-Path $PSScriptRoot "..\.."
$pagesDir = Join-Path $root "presentation\pages"
$pattern = '^(\p\d+|ps\d+(?:_d+)?|pb\d+(?:_d+)?)\.md$'

$invalid = @()
Get-ChildItem $pagesDir -Filter "*.md" -File | ForEach-Object {
    if ($_.Name -notmatch $pattern) {
        $invalid += $_.Name
    }
}

if ($invalid.Count -gt 0) {
    Write-Error ("Invalid page file name: " + ($invalid -join ", "))
    exit 1
}

Write-Host "[OK] Page ID validation passed."
exit 0
```

26. .claude/hooks/validate-page-schema.ps1

```
$ErrorActionPreference = "Stop"
$root = Join-Path $PSScriptRoot "..\.."
$pagesDir = Join-Path $root "presentation\pages"
$errors = @()

Get-ChildItem $pagesDir -Filter "*.md" -File | ForEach-Object {
    $content = Get-Content $_.FullName -Raw

    if ($content -notmatch '(?m)^id:\s*[\r\n]+' ) {
        $errors += "$($_.Name): missing id"
    }
    if ($content -notmatch '(?m)^type:\s*(cover|section|content)\s*$' ) {
        $errors += "$($_.Name): missing or invalid type"
    }
    if ($content -notmatch '(?m)^background:\s*[\r\n]+' ) {
        $errors += "$($_.Name): missing background"
    }
    if ($content -notmatch '(?m)^layout:\s*[\r\n]+' ) {
        $errors += "$($_.Name): missing layout"
    }
}

if ($errors.Count -gt 0) {
    $errors | ForEach-Object { Write-Error $_ }
    exit 1
}

Write-Host "[OK] Page schema validation passed."
exit 0
```

27. .claude/hooks/validate-layout.ps1

```
$ErrorActionPreference = "Stop"
$root = Join-Path $PSScriptRoot "..\.."
$pagesDir = Join-Path $root "presentation\pages"
$maxX = 1920
$maxY = 1080
$errors = @()

Get-ChildItem $pagesDir -Filter "*.md" -File | ForEach-Object {
    $lines = Get-Content $_.FullName
    foreach ($line in $lines) {
        if ($line -match '^s*x:s*(\d+)') {
            $x = [int]$Matches[1]
            if ($x -lt 0 -or $x -gt $maxX) {
                $errors += "$($_.Name): x=$x out of range"
            }
        }
        if ($line -match '^s*y:s*(\d+)') {
            $y = [int]$Matches[1]
            if ($y -lt 0 -or $y -gt $maxY) {
                $errors += "$($_.Name): y=$y out of range"
            }
        }
    }
}

if ($errors.Count -gt 0) {
    $errors | ForEach-Object { Write-Error $_ }
    exit 1
}

Write-Host "[OK] Layout validation passed."
exit 0
```

28. .claude/hooks/validate-build.ps1

```
$ErrorActionPreference = "Stop"
$root = Join-Path $PSScriptRoot "..\.."
$htmlDir = Join-Path $root "output\html"
$index = Join-Path $htmlDir "index.html"
$data = Join-Path $htmlDir "presentation.json"

if (-not (Test-Path $index)) {
    Write-Error "Build validation failed: output/html/index.html not found."
    exit 1
}

if (-not (Test-Path $data)) {
    Write-Error "Build validation failed: output/html/presentation.json not found."
    exit 1
}

Write-Host "[OK] Build validation passed."
exit 0
```

29. presentation/config/document.md

```
# Document Configuration

## Canvas
- width: 1920px
- height: 1080px
- ratio: 16:9

## Content Grid
- width: 1440px
- height: 1080px
- x: 240px
- y: 0px

## Default Surface
- background: `surface-page`

## Coordinate System
- origin: top-left
- x: left → right
- y: top → bottom

## Position Priority
1. absolute x/y
2. compound alignment
3. simple alignment
4. layout default
```


30. presentation/config/page-types.md

Page Types

cover

용도: 표지

기본 ID 예: `p1`

기본 layout: `cover`

section

용도: 서론/본론 구분 페이지

기본 ID 예: `ps1`, `pb1`

기본 layout: `section`

content

용도: 실제 내용 페이지

기본 ID 예: `ps1\_1`, `pb1\_2`

기본 layout: `default`, `image-right`, `overlap-title-box`

Rule

페이지 유형은 콘텐츠 역할을 나타낸다.

배경색이나 이미지 위치 같은 스타일 정보는 Page Type 이름에 포함하지 않는다.

31. presentation/config/manifest.md

```
# Presentation Manifest
```

```
## Document
```

```
- size: 1920 × 1080  
- grid: 1440 × 1080
```

```
## Pages
```

| order | id | type | background | layout |
|-------|-------|---------|--------------|-------------------|
| 1 | p1 | cover | surface-page | cover |
| 2 | ps1 | section | surface-page | section |
| 3 | ps1_1 | content | surface-page | default |
| 4 | ps1_2 | content | surface-soft | image-right |
| 5 | ps1_3 | content | surface-page | default |
| 6 | pb1 | section | surface-page | section |
| 7 | pb1_1 | content | surface-page | default |
| 8 | pb1_2 | content | surface-page | overlap-title-box |
| 9 | pb1_3 | content | surface-soft | image-right |

```
## Rule
```

페이지 순서, ID, 기본 유형은 이 파일을 기준으로 한다.

32. presentation/design-system/typography.md

Typography Tokens

| token | size | line-height | color | use |
|-------|------|-------------|----------------|---------|
| title | 96px | 110% | text-primary | 표지 대제목 |
| h1 | 60px | 120% | text-primary | 1 단계 제목 |
| h2 | 48px | 120% | text-primary | 2 단계 제목 |
| h3 | 36px | 132% | text-primary | 3 단계 제목 |
| body1 | 36px | 140% | text-primary | 큰 본문 |
| body3 | 24px | 134% | text-secondary | 일반 본문 |
| t1 | 32px | 130% | #000000 | 보조 제목 |
| t2 | 24px | 134% | #565656 | 보조 부제목 |

Rule

페이지 파일은 가능한 한 위 token 이름만 참조한다.

33. presentation/design-system/colors.md

Color Tokens

| token | value | use |
|----------------|---------|--------------|
| surface-page | #FFFFFF | 기본 페이지 배경 |
| surface-soft | #F1F1F1 | 연한 회색 페이지 배경 |
| surface-strong | #111111 | 어두운 면 |
| text-primary | #000000 | 기본 제목/본문 |
| text-secondary | #565656 | 보조 텍스트 |
| text-muted | #8A8A8A | 약한 정보 |
| border-default | #D9D9D9 | 기본 선 |
| accent-primary | #315EFB | 강조 색상 예시 |

Current Page Exception

- `ps1\_2`: surface-soft
- `pb1\_3`: surface-soft

34. presentation/design-system/spacing.md

Spacing Tokens

| token | value |
|---------|-------|
| space-1 | 8px |
| space-2 | 16px |
| space-3 | 24px |
| space-4 | 32px |
| space-5 | 48px |
| space-6 | 64px |
| space-7 | 96px |
| space-8 | 128px |

Rule

- 동일 정보 그룹은 작은 간격을 사용한다.
- 다른 정보 그룹은 더 큰 간격을 사용한다.
- 새로운 값 추가 전 기존 spacing token 으로 해결 가능한지 확인한다.

35. presentation/design-system/image-boxes.md

Image Box Tokens

Width Presets

| token | width |
|---------|-------|
| img-xl | 640px |
| img-lg | 420px |
| img-md | 300px |
| img-sm | 220px |
| img-xs | 180px |
| img-2xs | 150px |

Fixed Boxes

| token | width | height |
|---------|-------|--------|
| ImgBox1 | 952px | 1080px |
| ImgBox2 | 824px | 824px |
| ImgBox3 | 512px | 678px |

Rule

이미지 비율을 유지한 상태에서 token 크기를 우선 적용한다.

36. presentation/design-system/positions.md

Position Tokens

Simple Alignment

- left
- center
- right
- top
- bottom

Compound Alignment

- top-left
- top-right
- center-center
- bottom-left
- bottom-right

Absolute Position Examples

```
``yaml
position:
  x: 681
  y: 0
``
```

```
``yaml
position:
  x: 173
  y: 151
``
```

Priority

absolute X/Y > compound alignment > simple alignment > layout default

37. presentation/design-system/layouts.md

Layout Tokens

cover

- 대형 제목 중심
- 기본 배경 surface-page
- title token 사용

section

- 섹션 이름 중심
- 콘텐츠 최소화
- h1 또는 title 사용

default

- grid 내부 좌측 시작점 기준
- 제목 + 본문 구조

image-right

- 텍스트: 좌측
- 이미지: 우측
- 이미지 token 기본값: img-xl

overlap-title-box

큰 상자 위에 큰 제목이 일부 겹치는 구성.

```
```yaml
```

```
box:
 z-index: 1
```

```
title:
 z-index: 2
 overlap-y: 48px
```
```


38. presentation/pages/p1.md

```
----
id: p1
type: cover
background: surface-page
layout: cover
----

# p1

## title
``yaml
id: p1.title
component: title
text: Claude CLI PPT Automation
position:
  x: 240
  y: 340
``

## subtitle
``yaml
id: p1.subtitle
component: t2
text: Rules · Subagents · Skills · Hooks
position:
  x: 246
  y: 510
``
```

39. presentation/pages/ps1.md

```
----
id: ps1
type: section
background: surface-page
layout: section
----

# ps1

## title
``yaml
id: ps1.title
component: title
text: 서론
position:
  x: 240
  y: 390
``

## subtitle
``yaml
id: ps1.subtitle
component: t2
text: 프로젝트 구조와 기본 규칙
position:
  x: 246
  y: 540
``
```

40. presentation/pages/ps1_1.md

```
---
id: ps1_1
type: content
background: surface-page
layout: default
---

# ps1_1

## title
``yaml
id: ps1_1.title
component: h1
text: PPT 자동화 구조
position:
  x: 240
  y: 140
``,

## body01
``yaml
id: ps1_1.body01
component: body3
text: 규칙, 역할, 작업 절차, 자동 검증을 분리하여 관리합니다.
position:
  x: 240
  y: 280
``,
```

41. presentation/pages/ps1_2.md

```
----
id: ps1_2
type: content
background: surface-soft
layout: image-right
----

# ps1_2

## title
```yaml
id: ps1_2.title
component: h1
text: 프롬프트 엔지니어링
position:
 x: 173
 y: 151
```

## body01
```yaml
id: ps1_2.body01
component: body3
text: AI가 수행해야 할 역할과 조건을 구조적으로 정의합니다.
align: left
```

## image01
```yaml
id: ps1_2.image01
component: img-xl
source: ../assets/images/prompt-example.jpg
align: right
```
```

42. presentation/pages/ps1_3.md

```
---
id: ps1_3
type: content
background: surface-page
layout: default
---

# ps1_3

## title
``yaml
id: ps1_3.title
component: h1
text: Page ID 와 Element ID
position:
  x: 240
  y: 150
``,

## body01
``yaml
id: ps1_3.body01
component: body3
text: 페이지와 내부 요소를 분리하여 특정 대상만 안전하게 수정합니다.
position:
  x: 240
  y: 300
``,
```

43. presentation/pages/pb1.md

```
---
id: pb1
type: section
background: surface-page
layout: section
---

# pb1

## title
``yaml
id: pb1.title
component: title
text: 본론
position:
  x: 240
  y: 390
``

## subtitle
``yaml
id: pb1.subtitle
component: t2
text: 자동화 파이프라인 구축
position:
  x: 246
  y: 540
``
```

44. presentation/pages/pb1_1.md

```
---
id: pb1_1
type: content
background: surface-page
layout: default
---

# pb1_1

## title
``yaml
id: pb1_1.title
component: h1
text: Subagent
position:
  x: 240
  y: 150
``

## body01
``yaml
id: pb1_1.body01
component: body3
text: Planner, Designer, Builder, Reviewer 역할을 분리합니다.
position:
  x: 240
  y: 300
``
```

45. presentation/pages/pb1_2.md

```
----
id: pb1_2
type: content
background: surface-page
layout: overlap-title-box
----
```

```
# pb1_2
```

```
## box01
``yaml
id: pb1_2.box01
width: 1180
height: 520
position:
  x: 500
  y: 330
surface: surface-soft
````
```

```
title
``yaml
id: pb1_2.title
component: title
text: Skill
position:
 x: 300
 y: 250
z-index: 2
````
```

```
## body01
``yaml
id: pb1_2.body01
component: body3
text: 반복 가능한 작업 절차를 skill 로 정의합니다.
position:
  x: 600
  y: 520
````
```



## 46. presentation/pages/pb1\_3.md

```

id: pb1_3
type: content
background: surface-soft
layout: image-right

pb1_3

title
\`\`yaml
id: pb1_3.title
component: h1
text: Hook
position:
 x: 173
 y: 151
\`\`

body01
\`\`yaml
id: pb1_3.body01
component: body3
text: 파일 수정 이후 ID 와 Schema 를 자동으로 검사합니다.
position:
 x: 173
 y: 310
\`\`

image01
\`\`yaml
id: pb1_3.image01
component: img-lg
source: ../assets/images/hook-flow.jpg
align: right
\`\`
```

## 47. renderer/templates/cover.html

```
<section class="slide slide--cover" data-page-id="{{page.id}}">
 <div class="slide_grid">
 <h1 class="title" data-element-id="{{page.id}}.title">
 {{title.text}}
 </h1>

 <p class="t2" data-element-id="{{page.id}}.subtitle">
 {{subtitle.text}}
 </p>
 </div>
</section>
```

## 48. renderer/templates/section.html

```
<section class="slide slide--section" data-page-id="{{page.id}}">
 <div class="slide_grid">
 <header class="section-heading">
 <h1 class="title" data-element-id="{{page.id}}.title">
 {{title.text}}
 </h1>
 <p class="t2" data-element-id="{{page.id}}.subtitle">
 {{subtitle.text}}
 </p>
 </header>
 </div>
</section>
```

## 49. renderer/templates/content.html

```
<section class="slide slide--content {{page.layout}}" data-page-id="{{page.id}}">
 <div class="slide_grid">
 <h1 class="h1" data-element-id="{{page.id}}.title">
 {{title.text}}
 </h1>

 <div class="content-body" data-element-id="{{page.id}}.body01">
 {{body01.text}}
 </div>

 {{#if image01}}

 {{/if}}
 </div>
</section>
```

## 50. renderer/css/tokens.css

```
:root {
 --document-width: 1920px;
 --document-height: 1080px;
 --grid-width: 1440px;
 --grid-left: 240px;

 --surface-page: #ffffff;
 --surface-soft: #f1f1f1;
 --text-primary: #000000;
 --text-secondary: #565656;

 --font-title: 96px;
 --font-h1: 60px;
 --font-h2: 48px;
 --font-h3: 36px;
 --font-body1: 36px;
 --font-body3: 24px;

 --img-xl: 640px;
 --img-lg: 420px;
 --img-md: 300px;
}
```

## 51. renderer/css/components.css

```
* {
 box-sizing: border-box;
}

body {
 margin: 0;
 font-family: Arial, "Noto Sans KR", sans-serif;
 background: #222;
}

.title {
 margin: 0;
 font-size: var(--font-title);
 line-height: 1.1;
 color: var(--text-primary);
}

.h1 {
 margin: 0;
 font-size: var(--font-h1);
 line-height: 1.2;
}

.t2 {
 font-size: 24px;
 line-height: 1.34;
 color: var(--text-secondary);
}

.content-body {
 font-size: var(--font-body3);
 line-height: 1.34;
}

.slide-image {
 display: block;
 object-fit: cover;
}
```

## 52. renderer/css/slides.css

```
.slide {
 position: relative;
 width: var(--document-width);
 height: var(--document-height);
 overflow: hidden;
 background: var(--surface-page);
 page-break-after: always;
}

.slide[data-background="surface-soft"],
.slide.surface-soft {
 background: var(--surface-soft);
}

.slide__grid {
 position: relative;
 width: var(--grid-width);
 height: 100%;
 margin-left: var(--grid-left);
}

.image-right .slide-image {
 position: absolute;
 width: var(--img-xl);
 right: 0;
 top: 180px;
}

.overlap-title-box .title {
 position: relative;
 z-index: 2;
}

@media print {
 body { background: transparent; }
 .slide { break-after: page; }
}
```

## 53. renderer/js/presentation.js

```
async function loadPresentation() {
 const response = await fetch('./presentation.json');
 if (!response.ok) {
 throw new Error('presentation.json을 불러오지 못했습니다.');
```

}

```
 return response.json();
}

function escapeHtml(value = '') {
 return value
 .replaceAll('&', '&')
 .replaceAll('<', '<')
 .replaceAll('>', '>');
}

function renderRawPage(page) {
 return `
 <section class="slide" data-page-file="${page.file}">
 <div class="slide_grid">
 <pre>${escapeHtml(page.markdown)}</pre>
 </div>
 </section>
 `;
}

async function init() {
 const root = document.querySelector('#presentation');
 const pages = await loadPresentation();
 root.innerHTML = pages.map(renderRawPage).join('');
}

init().catch((error) => {
 console.error(error);
 document.body.innerHTML = `<pre>${escapeHtml(error.message)}</pre>`;
});
```



## 54. renderer/index.html

```
<!doctype html>
<html lang="ko">
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Claude CLI PPT Automation</title>

 <link rel="stylesheet" href="./css/tokens.css">
 <link rel="stylesheet" href="./css/components.css">
 <link rel="stylesheet" href="./css/slides.css">
</head>
<body>
 <main id="presentation" aria-label="Presentation"></main>
 <script src="./js/presentation.js"></script>
</body>
</html>
```

## 55. scripts/build.ps1

```
$ErrorActionPreference = "Stop"
$root = Split-Path $PSScriptRoot -Parent
Set-Location $root

Write-Host "[1/4] Validate"
& powershell -NoProfile -ExecutionPolicy Bypass -File ".\scripts\validate.ps1"
if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }

Write-Host "[2/4] Prepare output/html"
$htmlOut = Join-Path $root "output/html"
if (Test-Path $htmlOut) {
 Get-ChildItem $htmlOut -Force | Remove-Item -Recurse -Force
}
else {
 New-Item -ItemType Directory -Force -Path $htmlOut | Out-Null
}

Write-Host "[3/4] Copy renderer"
Copy-Item ".\renderer*" $htmlOut -Recurse -Force

$pages = Get-ChildItem ".\presentation\pages*.md" -File |
 Sort-Object Name |
 ForEach-Object {
 [ordered]@{
 file = $_.Name
 markdown = Get-Content $_.FullName -Raw
 }
 }

$pages | ConvertTo-Json -Depth 5 |
 Set-Content (Join-Path $htmlOut "presentation.json") -Encoding UTF8

Write-Host "[4/4] Validate build"
& powershell -NoProfile -ExecutionPolicy Bypass -File ".\.claude\hooks\validate-build.ps1"
exit $LASTEXITCODE
```

## 56. scripts/export-pdf.ps1

```
$ErrorActionPreference = "Stop"
$root = Split-Path $PSScriptRoot -Parent
$html = Join-Path $root "output\html\index.html"
$pdf = Join-Path $root "output\pdf\presentation.pdf"

$chromeCandidates = @(
 "$env:ProgramFiles\Google\Chrome\Application\chrome.exe",
 "${env:ProgramFiles(x86)}\Google\Chrome\Application\chrome.exe"
)

$chrome = $chromeCandidates | Where-Object { Test-Path $_ } | Select-Object -First 1
if (-not $chrome) {
 Write-Error "Google Chrome 실행 파일을 찾을 수 없습니다."
 exit 1
}

if (-not (Test-Path $html)) {
 Write-Error "먼저 scripts/build.ps1 을 실행하세요."
 exit 1
}

$url = "file:/// " + ($html -replace '\\', '/')
& $chrome --headless --disable-gpu --print-to-pdf="$pdf" $url

if (-not (Test-Path $pdf)) {
 Write-Error "PDF 생성 실패"
 exit 1
}

Write-Host "[OK] PDF: $pdf"
```

## 57. scripts/validate.ps1

```
$ErrorActionPreference = "Stop"
$root = Split-Path $PSScriptRoot -Parent
$validators = @(
 ".claude\hooks\validate-page-id.ps1",
 ".claude\hooks\validate-page-schema.ps1",
 ".claude\hooks\validate-layout.ps1"
)

foreach ($validator in $validators) {
 $path = Join-Path $root $validator
 Write-Host "Running $validator"
 & powershell -NoProfile -ExecutionPolicy Bypass -File $path

 if ($LASTEXITCODE -ne 0) {
 Write-Error "Validation failed: $validator"
 exit $LASTEXITCODE
 }
}

Write-Host "[OK] All validations passed."
exit 0
```